ENERGETIKAI TECHNIKUM ÉS KOLLÉGIUM

Vizsgaremek

Könyvhuszárok

Online könyvesbolt webshop

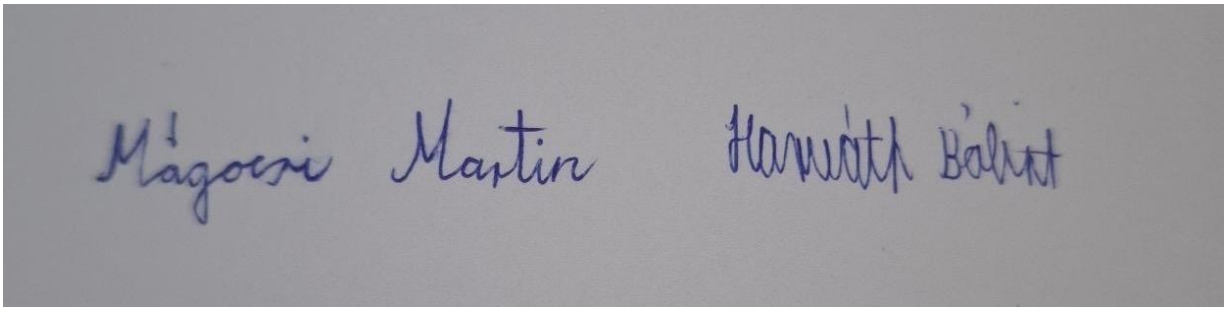
Szakma:	Készítették:
Szoftverfejlesztő és -tesztelő	Horváth Bálint Zoltán
5 0613 12 03	Mágocsi Martin

2026.

Nyilatkozat

Alulírott Horváth Bálint Zoltán és Mágocsi Martin büntetőjogi felelősségem teljes tudatában nyilatkozom arról, hogy az itt szereplő vizsgaremek csoportmunka eredménye és sem részeiben sem egészében nem került még kereskedelmi forgalomba, ill. publikálásra, a GPL licenszelésű programrészek kivételével.

Paks, 2026. április 22.



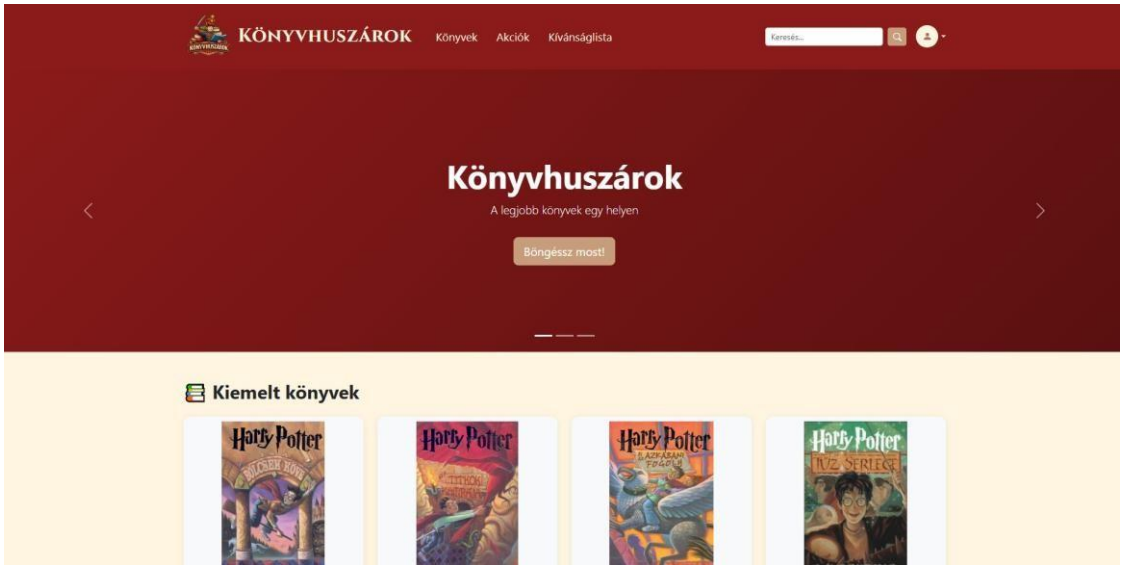
Tartalomjegyzék

I. Bevezetés 4
II. Témaválasztás indoklása 6
III. Fejlesztői dokumentáció, technikai leírás 7
1. Specifikáció 7
2. Az alkalmazott fejlesztői eszközök 8
3. Adatmodell leírása 9
4. Algoritmusok 10
5. Tesztelési dokumentáció 15
IV. Felhasználói dokumentáció 22
1. A program általános specifikációja, célja 22
2. Rendszerkövetelmények 22
3. A program telepítése 23
4. A program használatának részletes leírása 24
V. Összegzés 29
VI. Ábrajegyzék 30
VII. Irodalomjegyzék, hivatkozásjegyzék 31

I. Bevezetés

A Könyvhuszárok projekt egy modern, teljes funkcionalitású online könyvesbolt webalkalmazást valósít meg, amelyet a szakmai vizsga keretein belül készítettünk. Az alkalmazás lehetővé teszi a felhasználók számára, hogy könyveket böngésszenek, keressenek, kosarukba helyezzenek és megrendeljék azokat, miközben az adminisztrátorok teljes körű felügyelettel rendelkeznek a termékek, rendelések és felhasználók kezelésére.

A webalkalmazás napjainkban egyre fontosabb szerepet játszik a kereskedelemben. Az online vásárlás kényelme és elérhetősége miatt egyre többen választják ezt a formát a hagyományos, fizikai boltok látogatása helyett. A könyvpiac sem kivétel ez alól – a digitális platformokon keresztül történő könyvvásárlás világszerte dinamikusan növekszik, és Magyarországon is egyre nagyobb teret hódít. Projektünk célja egy valós piaci igényeket kielégítő megoldás létrehozása volt, amely mind a vevők, mind az adminisztrátorok számára átlátható és könnyen használható felületet nyújt.



1. ábra:A Könyvhuszárok főoldala

Az alkalmazás fejlesztése során arra törekedtünk, hogy egy olyan rendszert hozzunk létre, amely nemcsak funkcionálisan teljes körű, hanem technikai szempontból is korszerű megoldásokat alkalmaz. A backend oldalon ASP.NET Core 8.0 keretrendszert és RESTful API architektúrát alkalmaztunk, amely lehetővé teszi a rendszer könnyű bővíthetőségét és karbantarthatóságát. Az adatokat MySQL relációs adatbázisban tároljuk, amelyet Entity Framework Core ORM segítségével kezelünk. A felhasználói felület Bootstrap 5 keretrendszer segítségével reszponzív kialakítást kapott, amely egyaránt jól működik asztali számítógépen, tableten és okostelefonon is.

A biztonságos hozzáférés érdekében JWT (JSON Web Token) alapú autentikációt vezettünk be, amely iparági szabványnak számít a modern webalkalmazásokban. A jelszavak bcrypt algoritmussal titkosítva kerülnek tárolásra, így a felhasználók adatai védve vannak. A rendszer kétféle szerepkört különböztet meg: a hagyományos felhasználót, aki vásárolhat és kezelheti saját fiókját, illetve az adminisztrátort, aki teljes hozzáféréssel rendelkezik a rendszer összes funkciójához.

A fejlesztés során kiemelt figyelmet fordítottunk a clean code elvek alkalmazására. A backend kódot Services és DTOs rétegekre bontottuk, amelyek biztosítják az üzleti logika és a HTTP kérések kezelésének szétválasztását. A Single Responsibility Principle érvényesítése érdekében minden osztály kizárólag egy jól meghatározott feladatot lát el. A frontend oldalon szintén törekedtünk a kód újrafelhasználhatóságára: a közös segédfüggvényeket (kosárba adás, értesítések megjelenítése, könyv kártyák generálása) egy önálló utils.js fájlba szerveztük ki, ezzel megvalósítva a DRY (Don't Repeat Yourself) elvet.

Az alkalmazás funkcionalitása túlmutat egy egyszerű tanulmányi projekten: a rendszer valós üzleti igényeket elégít ki. A felhasználók böngészhetnek a könyvek között, szűrhetnek kategória és ár szerint, kívánságlistát kezelhetnek, értékeléseket írhatnak, valamint akciós könyveket tekinthetnek meg egy dedikált Akciók oldalon. Az adminisztrátorok teljes körű termékkezelési, rendeléskövetési és felhasználókezelési lehetőségekkel rendelkeznek, beleértve a könyvenkénti kedvezmények beállítását is. A projekt fejlesztése során mindketten jelentős tapasztalatot szereztünk a modern webes technológiák alkalmazásában, amelyet a jövőbeli szakmai munkánk során is hasznosítani kívánunk.

II. Témaválasztás indoklása

Az e-kereskedelem az elmúlt évtizedben robbanásszerűen fejlődött, és a könyvpiac sem kivétel ez alól. Egy online könyvesbolt fejlesztése kiválóan alkalmas arra, hogy bemutassuk a modern webes technológiák használatát, különösen egy RESTful API alapú, adatbázis-vezérelt alkalmazás esetében.

A témaválasztást az alábbi szempontok indokolják:

Téma aktualitása: Az online vásárlás – különösen a könyvek területén – évről évre növekvő tendenciát mutat. A fizikai könyvesboltok forgalma csökken, míg az internetes rendelések száma folyamatosan emelkedik. Egy jól megtervezett webshop tehát valós piaci igényt elégít ki, nem csupán elméleti feladat megoldását jelenti.

Technikai komplexitás: A projekt lehetőséget nyújt a backend és frontend fejlesztés egyidejű alkalmazására. A feladat magában foglalja az adatbázis tervezését, a szerver oldali logika megírását, a REST API végpontok kialakítását, a felhasználói felület megvalósítását, valamint a kettő közötti kommunikáció biztosítását. Ez a sokrétűség ideális vizsgáló környezetet teremt a megszerzett ismeretek bemutatásához.

Tanult technológiák hasznosítása: A projekt lehetőséget adott az iskolai tanulmányok során elsajátított ismeretek – ASP.NET Core, MySQL, HTML/CSS/JavaScript – valós környezetben történő alkalmazására. A fejlesztés során nemcsak az elméleti tudást tudtuk kamatoztatni, hanem számos új, gyakorlati problémával is szembesültünk, amelyek megoldása jelentősen fejlesztette programozói gondolkodásunkat.

Skálázhatóság és bővíthetőség: A rendszer modulárisan épül fel, így a jövőben könnyen bővíthető további funkciókkal, például online fizetési rendszerrel, ajánlórendszerrel vagy mobilalkalmazással.

A fejlesztéshez az alábbi eszközöket és eljárásokat alkalmaztuk: RESTful API architektúrát, Entity Framework Core ORM keretrendszert az adatbázis kezeléséhez, JWT token alapú authentikációt, valamint Bootstrap 5 reszponzív frontend keretrendszert.

III. Fejlesztői dokumentáció, technikai leírás

1. Specifikáció

A Könyvhuszárok webalkalmazással szemben támasztott követelmények az alábbiak szerint összegezhetők. A rendszer egyaránt kiszolgálja a hétköznapi vásárlókat és az adminisztrátorokat, biztosítva minden szükséges funkciót az üzleti folyamatok zökkenőmentes lebonyolításához.

Funkcionális követelmények – Felhasználói oldal:

- Regisztráció és bejelentkezés email cím + jelszó alapján
- Könyvek böngészése, szűrése kategória, szerző és ár szerint
- Keresés cím és szerző alapján
- Könyvek részletes megtekintése (leírás, ár, készlet, értékelések)
- Kosár kezelése (hozzáadás, módosítás, törlés)
- Rendelés leadása szállítási címmel és fizetési móddal
- Rendelési előzmények megtekintése
- Profil szerkesztése, jelszó megváltoztatása
- Kívánságlista kezelése
- Könyvek értékelése (1-5 csillag + szöveges vélemény)

Funkcionális követelmények – Adminisztrátori oldal:

- Könyvek létrehozása, szerkesztése, törlése
- Rendelések megtekintése és státuszának frissítése
- Felhasználók kezelése (tiltás, aktiválás, szerepkör módosítás)
- Kategóriák kezelése
- Dashboard statisztikák

Nem funkcionális követelmények:

- Reszponzív dizájn (mobil, tablet, asztali nézet)
- JWT token alapú biztonságos autentikáció
- RESTful API architektúra
- Swagger API dokumentáció
- Jelszavak bcrypt titkosítása

2. Az alkalmazott fejlesztői eszközök

Backend fejlesztés:

- ASP.NET Core 8.0 Web API – a szerver oldali alkalmazás keretrendszere
- C# programozási nyelv – a backend logika megvalósításához
- Entity Framework Core 8.0 – ORM keretrendszer az adatbázis kezeléséhez
- Pomelo.EntityFrameworkCore.MySql 8.0
- Microsoft.AspNetCore.Authentication.JwtBearer 8.0 – JWT token kezelés
- BCrypt.Net-Next 4.0 – jelszó titkosítás
- Swashbuckle.AspNetCore 6.5 – Swagger API dokumentáció

Frontend fejlesztés:

- HTML5, CSS3, Vanilla JavaScript – az alkalmazás felhasználói felülete
- Bootstrap 5.3 – reszponzív dizájn keretrendszer
- Bootstrap Icons 1.10 – ikonkészlet

Adatbázis:

- MySQL 8.0 – relációs adatbázis-kezelő rendszer
- XAMPP – helyi fejlesztői környezet (Apache + MySQL + phpMyAdmin)
- phpMyAdmin – adatbázis grafikus kezelő felület

Fejlesztői környezet:

- Visual Studio 2022 – C# backend fejlesztéshez
- Visual Studio Code – HTML/CSS/JavaScript frontend fejlesztéshez
- Git – verziókezelés
- GitHub – forráskód megosztás és verziókövetés

Dokumentáció:

- Microsoft Word 2016 – dokumentáció készítése
- draw.io – ER diagram készítése
- dbdiagram.io – adatbázis diagram készítés

3. Adatmodell leírása

Az alkalmazás MySQL adatbázist használ, amelyet Entity Framework Core Code-First megközelítéssel hoztunk létre. Az adatbázis 13 táblából áll, amelyek a webshop összes adatát tartalmazzák.

Főbb adattáblák:

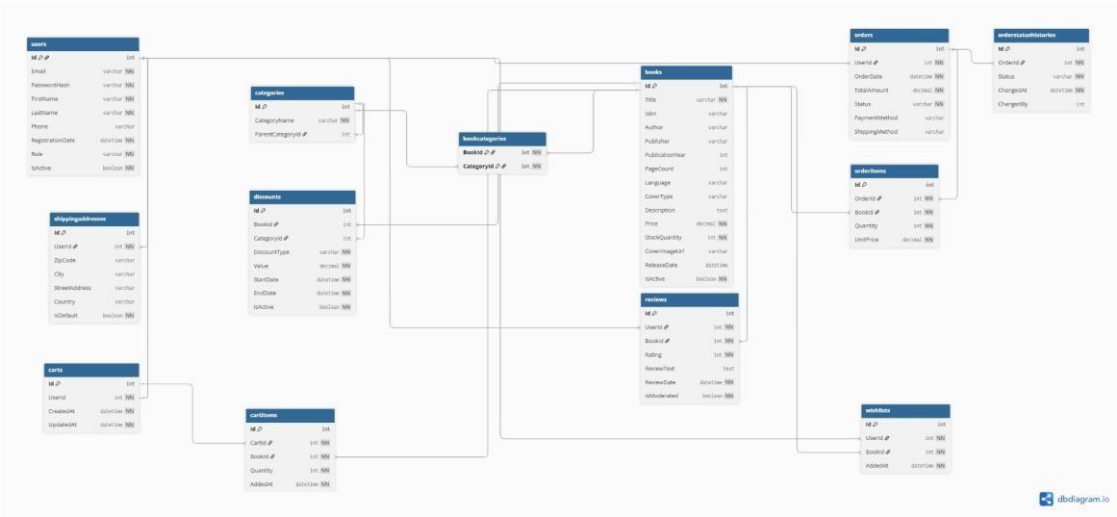
- users – A regisztrált felhasználók adatait tárolja (név, email, jelszó, szerepkör, aktív státusz)
- books – A könyvek adatait tárolja (cím, szerző, ár, készlet, leírás, borítókép URL)
- categories – Könyvkategóriák hierarchikus tárolása (főkategória és alkategória)
- book_categories – Több-többhöz kapcsolótábla könyvek és kategóriák között
- carts – Felhasználók vásárlókosarainak adatai (egy felhasználóhoz egy kosár tartozik)
- cartitems – A kosárban lévő könyvek és mennyiségük
- orders – Leadott rendelések főadatai (összeg, státusz, fizetési mód, szállítási mód)
- orderitems – A rendelésben szereplő könyvek és egységáraik •

  orderstatushistories – Rendelések státuszváltozásainak előzményei
- shippingaddresses – Felhasználók mentett szállítási címei
- reviews – Könyvek felhasználói értékelései (1-5 csillag + szöveges vélemény)
- wishlists – Felhasználók kívánságlistájának elemei

A táblák közötti főbb kapcsolatok:

- users 1→N orders – egy felhasználónak több rendelése lehet
- users 1↔1 carts – minden felhasználóhoz egy kosár tartozik
- orders 1→N orderitems – egy rendelés több tételt tartalmaz
- books N↔N categories – egy könyv több kategóriában is szerepelhet
- orders 1→N orderstatushistories – egy rendelésnek több státusz előzménye lehet
- books 1→N reviews – egy könyvhöz több értékelés tartozhat
- books 1→N wishlists – egy könyv több felhasználó kívánságlistáján szerepelhet

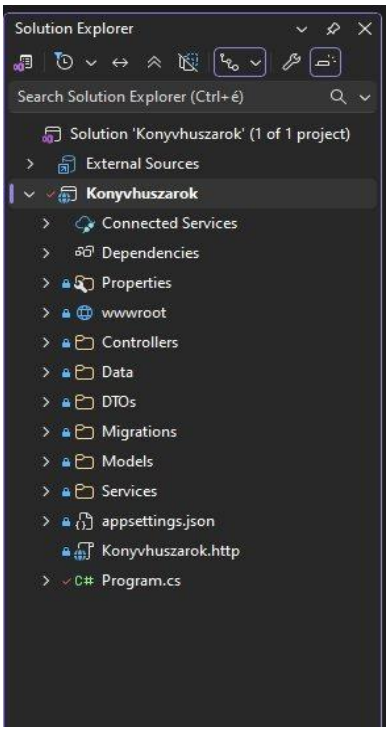
Az adatbázis diagramja:



2. ábra:A Könyvhuszárok adatbázis diagramja

4. Algoritmusok

Az alábbiakban bemutatjuk az alkalmazás legfontosabb algoritmikus megoldásait. Az algoritmusok leírásánál feltüntetjük az érintett fájlt, a bemeneti paramétereket, a visszatérési értéket és a működési logikát. A rendszer összesen 9 Controller osztályból áll, amelyek mindegyike egy-egy funkcióterületet fed le.



3. ábra: A projekt mappastruktúrája Visual Studio Solution Explorerben

1. JWT Token generálás és bejelentkezés (AuthController.cs)

Végpont: POST /api/auth/login | Bemenet: LoginRequest (email, jelszó) |

Kimenet: JWT token + felhasználói adatok

- Email cím alapján megkeressük a felhasználót az adatbázisban
- BCrypt segítségével ellenőrizzük a megadott jelszót a hash-elt változattal szemben
- Ellenőrizzük, hogy a fiók aktív-e (nincs letiltva)
- Claims listát hozunk létre (ID, Email, Role, Név)
- HMAC-SHA256 algoritmussal aláírjuk a tokent a titkos kulccsal
- Visszaküldjük a tokent a felhasználó adataival együtt

A frontend az így kapott tokent a localStorage-ban tárolja, és minden védett API hívásnál elküldi az Authorization: Bearer {token} fejlécben. A token 24 óra után automatikusan lejár, ekkor a rendszer 401 Unauthorized választ küld, és a felhasználót visszairányítja a bejelentkezési oldalra. A regisztrációnál a jelszó BCrypt algoritmussal kerül hash-elésre (cost factor: 11), így az adatbázisban soha nem tárolódik nyílt szövegű jelszó.

```
// Bejelentkezés
1 reference
public async Task<AuthResult> LoginAsync(LoginRequest request)
{
    var user = await _context.Users.FirstOrDefaultAsync(u => u.Email == request.Email);

    if (user == null || !BCrypt.Net.BCrypt.Verify(request.Password, user.PasswordHash))
        return new AuthResult { Success = false, Message = "Hibás email vagy jelszó." };

    if (!user.IsActive)
        return new AuthResult { Success = false, Message = "Ez a fiók le van tiltva." };

    var token = GenerateJwtToken(user);
    var userData = new
    {
        user.Id,
        user.Email,
        user.FirstName,
        user.LastName,
        user.Role
    };

    return new AuthResult { Success = true, Message = "Sikeres bejelentkezés.", Token = token, UserData = userData };
}
```

4. ábra:A LoginAsync metódus forráskódja (AuthService.cs)

2. Rendelés leadása (OrdersController.cs)

Végpont: POST /api/orders | Bemenet: CreateOrderRequest (fizetési mód, szállítási mód) | Kimenet: rendelés azonosítója és végösszege

A rendelés leadása több lépésből álló folyamat, amely biztosítja az adatok konzisztenciáját:

- A felhasználó kosarának betöltése az összes tétellel
- Készletellenőrzés minden tételre – ha nincs elegendő készlet, hibát adunk vissza
- Végösszeg kiszámítása (mennyiség × egységár)
- Order rekord létrehozása az adatbázisban
- OrderItem rekordok létrehozása minden kosár tételhez
- Készlet csökkentése a megrendelt mennyiséggel
- Státusz előzmény bejegyzés létrehozása („pending" állapot)

- Kosár ürítése a sikeres rendelés után

A rendelés leadása tranzakcionális logikát követ: ha bármelyik lépésben hiba történik (pl. nincs elegendő készlet), a rendszer hibaüzenetet küld vissza és nem módosítja az adatbázist. A rendelés státusza kezdetben "pending", amelyet az adminisztrátor később frissíthet a következő lépcsőkön keresztül: pending → paid → processing → shipped → delivered. Minden státuszváltás rögzítésre kerül az orderstatushistories táblában.

```
1 reference
public async Task<(bool Success, string Message, int OrderId, decimal TotalAmount)> CreateOrderAsync(int userId, string paymentMethod, string shippingMethod)
{
    var cart = await _context.Carts
        .Include(c => c.CartItems)
            .ThenInclude(ci => ci.Book)
        .FirstOrDefaultAsync(c => c.UserId == userId);

    if (cart == null || !cart.CartItems.Any())
        return (false, "A kosár üres.", 0, 0);

    foreach (var item in cart.CartItems)
    {
        if (item.Book.StockQuantity < item.Quantity)
            return (false, $"Nincs elegendő készlet: {item.Book.Title}", 0, 0);
    }

    var totalAmount = cart.CartItems.Sum(ci => ci.Book.Price * ci.Quantity);

    var order = new Order
    {
        UserId = userId,
        TotalAmount = totalAmount,
        Status = "pending",
        PaymentMethod = paymentMethod,
        ShippingMethod = shippingMethod,
        OrderDate = DateTime.UtcNow
    };

    _context.Orders.Add(order);
    await _context.SaveChangesAsync();

    foreach (var item in cart.CartItems)
    {
        _context.OrderItems.Add(new OrderItem
        {
            OrderId = order.Id,
            BookId = item.BookId,
            Quantity = item.Quantity,
            UnitPrice = item.Book.Price
        });
        item.Book.StockQuantity -= item.Quantity;
    }

    _context.OrderStatusHistories.Add(new OrderStatusHistory
```

5. ábra:A CreateOrderAsync metódus forráskódja (OrderService.cs)

3. Könyvek szűrése és lapozása (BooksController.cs)

Végpont: GET /api/books | Bemenet: search, categoryId, sortBy, page, pageSize (URL paraméterek) | Kimenet: könyvlista lapozási adatokkal

A könyvlistázó végpont több szűrési paramétert támogat egyidejűleg:

- search – cím és szerző szerinti keresés: a rendszer SQL LIKE lekérdezést használ, amely részleges egyezéseket is megtalál (pl. "harry" megtalálja a "Harry Potter" könyveket)
- categoryId – a rendszer a bookcategories kapcsolótáblán keresztül szűr: csak azokat a könyveket adja vissza, amelyek az adott kategóriához vannak rendelve
- sortBy – rendezési lehetőségek: price_asc (ár növekvő), price_desc (ár csökkenő), newest (legújabb ReleaseDate szerint), alapértelmezett: A-Z cím szerint
- page és pageSize – lapozás SKIP/TAKE megvalósítással. A válasz tartalmazza a teljes találatszámot és az oldalak számát is.

```
// GET: api/books
[HttpGet]
0 references
public async Task<IActionResult> GetBooks(
    [FromQuery] string? search,
    [FromQuery] int? categoryId,
    [FromQuery] string? sortBy,
    [FromQuery] int page = 1,
    [FromQuery] int pageSize = 12)
{
    var result = await _bookService.GetBooksAsync(search, categoryId, sortBy, page, pageSize);
    return Ok(result);
}
```

6. ábra:A GetBooks végpont forráskódja (BooksController.cs)

4. Kosár kezelése (CartController.cs)

Végpont: POST /api/cart/items | Bemenet: CartItemRequest (bookId, quantity) | Kimenet: sikerüzenet

A kosárkezelő algoritmus GetOrCreateCart segédfüggvényt használ, amely automatikusan létrehozza a kosarat ha még nem létezik a felhasználónak.
Kosárba adáskor a rendszer ellenőrzi a könyv elérhetőségét és a raktárkészletet. Ha a könyv már szerepel a kosárban, a mennyiség növekszik; egyébként új tételként kerül be. Mennyiség módosításakor 0 vagy negatív értéknél a tétel törlődik. A kosár frissítésekor az UpdatedAt mező időbélyege automatikusan frissül.

5. Felhasználói jogosultság-kezelés ([Authorize] attribútum)

A rendszer háromféle hozzáférési szintet különböztet meg. Nyilvános végpontok (pl. könyvlista, könyv részletek, bejelentkezés, regisztráció) token nélkül is elérhetők. Bejelentkezett felhasználói végpontok ([Authorize]) érvényes JWT tokent igényelnek – ide tartozik a kosár, rendelés, profil és kívánságlista kezelése. Adminisztrátori végpontok ([Authorize(Roles = "admin")]) csak admin szerepkörű felhasználó tokenjével érhetők el – ide tartozik a könyv CRUD, rendelés státusz frissítés és felhasználókezelés. A token érvényességét az ASP.NET Core middleware automatikusan ellenőrzi minden kérés feldolgozása előtt.

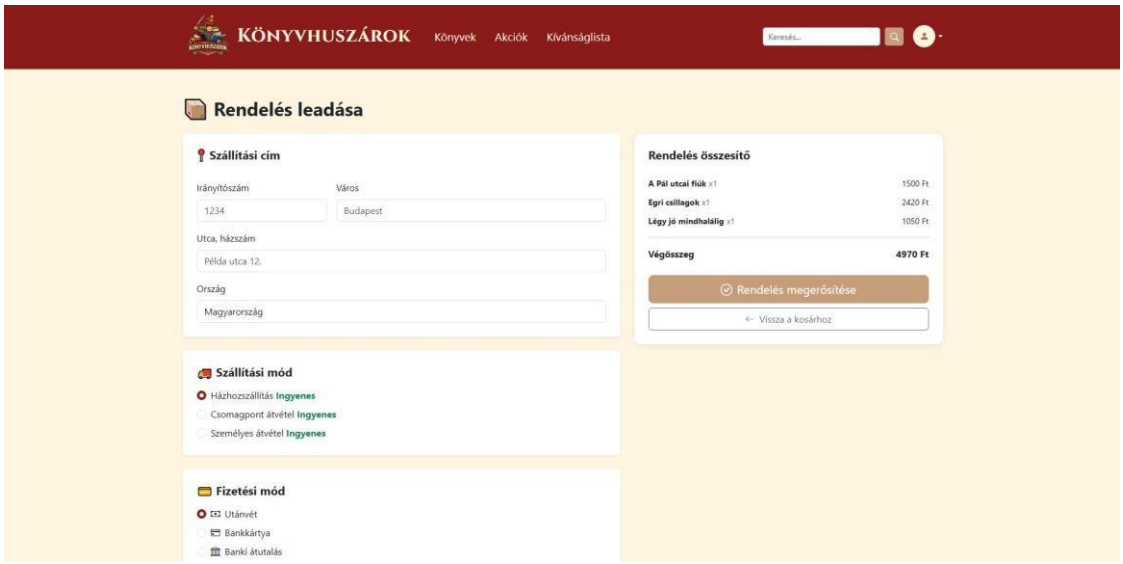
6. Értékelés kezelése (ReviewsController.cs)

Végpont: POST /api/reviews/book/{bookId} | Bemenet: ReviewRequest (rating, reviewText) | Kimenet: sikerüzenet

Értékelés leadásakor a rendszer ellenőrzi, hogy a felhasználó már értékelte-e az adott könyvet. Egy felhasználó egy könyvhöz csak egy értékelést írhat. A rating értéke 1 és 5 között kell legyen, ezt a rendszer validálja. Az értékelések lekérésekor a rendszer kiszámítja az átlagos értékelést és visszaadja az összes értékelést a felhasználó nevével együtt. Az átlag számítása: az összes rating összege osztva az értékelések számával, egy tizedesjegy pontossággal kerekítve.

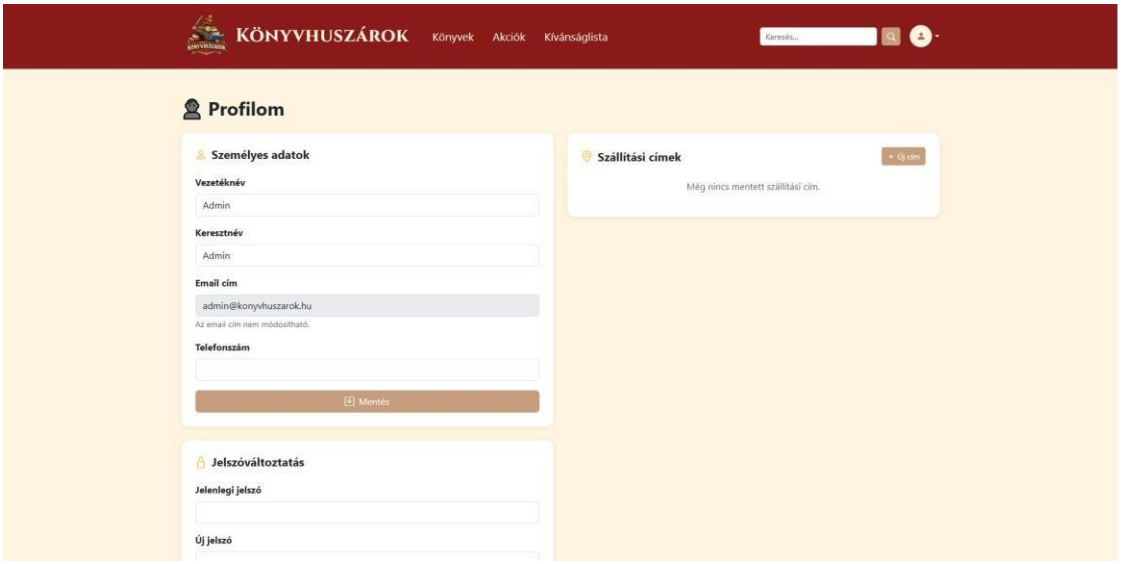
7. Szállítási cím kezelése (AddressesController.cs)

Egy felhasználónak több szállítási címe lehet, amelyek közül egy alapértelmezettként jelölhető meg. Új cím hozzáadásakor, ha az IsDefault értéke true, a rendszer automatikusan törli a többi cím alapértelmezett státuszát, így mindig csak egy cím lehet alapértelmezett. Ha ez az első cím a felhasználóhoz, automatikusan alapértelmezettként kerül mentésre. A checkout folyamat során az alapértelmezett cím automatikusan betöltődik az űrlapba. Az alábbi képen a rendelés leadása oldal látható a szállítási adatok kitöltésével:



7. ábra:Rendelés leadása oldal

A profil oldalon szerkeszthetők a személyes adatok és kezelhetők a szállítási címek:



8. ábra:Profil oldal

8. Frontend API kommunikáció (auth.js)

A frontend minden API kommunikációhoz az apiCall() segédfüggvényt használja, amely egységes módon kezeli a HTTP kéréseket. Ez a függvény automatikusan hozzáadja a JWT tokent az Authorization fejléchez, beállítja a Content-Type: application/json fejlécet, és kezeli a 401-es hibát (token lejárat esetén kijelentkeztet és visszairányít a bejelentkezési oldalra). A felhasználói adatok és a token a böngésző localStorage-ban kerülnek tárolásra, így az oldal frissítése után is bejelentkezve marad a felhasználó.

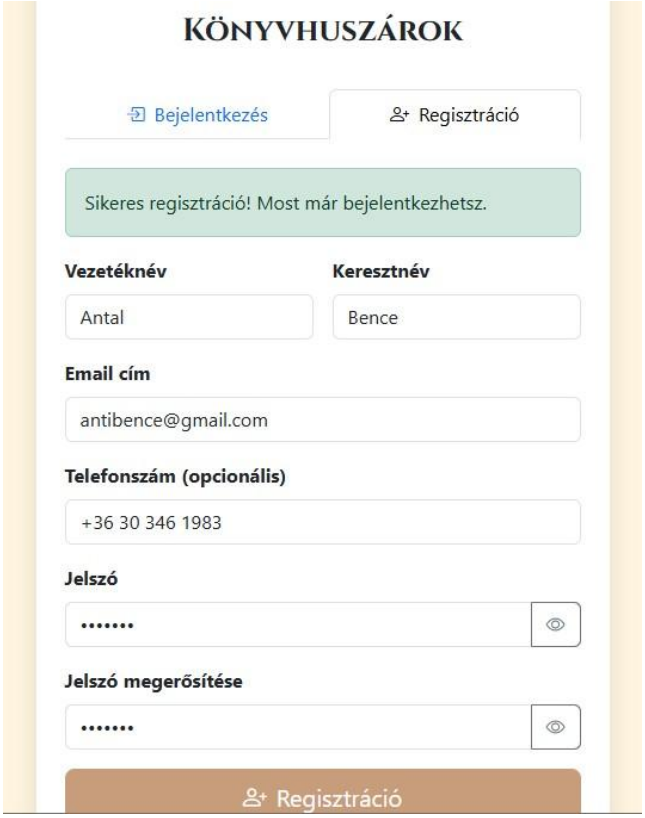
5. Tesztelési dokumentáció

A tesztelés során két fő módszert alkalmaztunk: a fekete doboz és a fehér doboz tesztelést.

Fekete doboz tesztelés

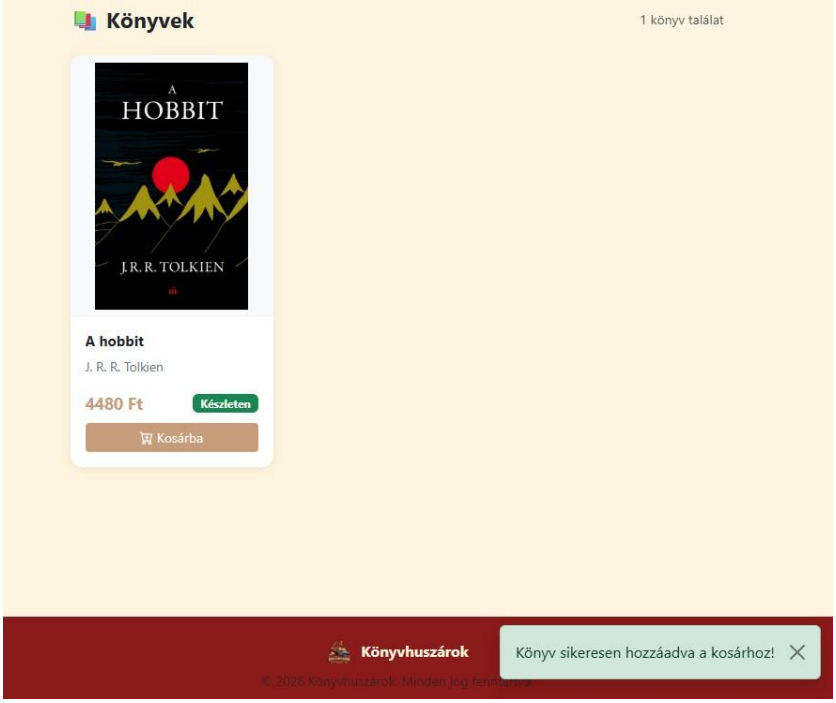
A felhasználói felületen keresztül teszteltük az alkalmazást, a belső megvalósítás ismerete nélkül. Az alábbi tesztesetek kerültek végrehajtásra:

- **1. teszteset – Regisztráció:** A regisztrációs űrlap kitöltése valós adatokkal, majd a Regisztráció gomb megnyomása. Várt eredmény: sikeres regisztráció üzenet. Eredmény: SIKERES ✓

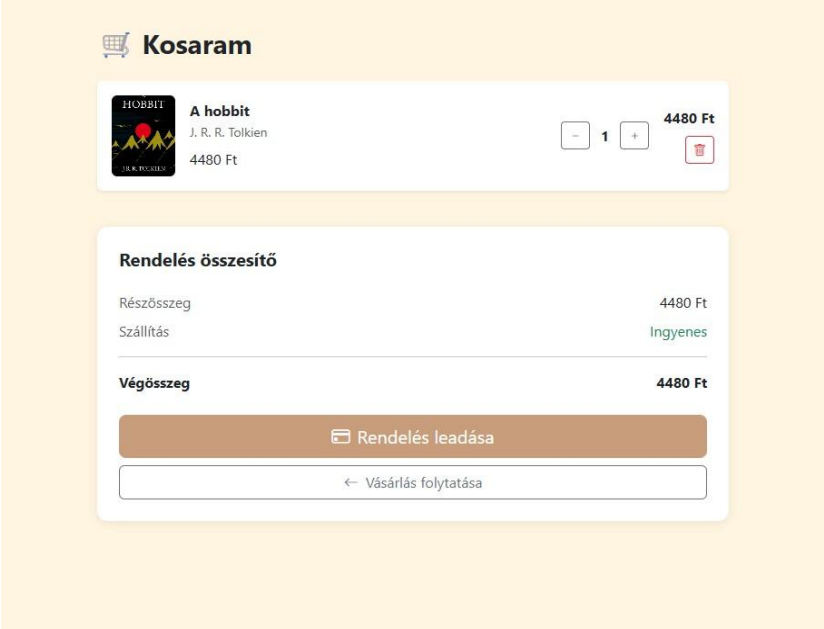


9. ábra:Sikeres regisztráció

- **2. teszteset – Hibás bejelentkezés:** Hibás jelszó megadása helyes email címmel. Várt eredmény: hibaüzenet megjelenik. Eredmény: SIKERES ✓
- **3. teszteset – Kosárba helyezés:** Bejelentkezés után könyv kosárba helyezése. Várt eredmény: könyv megjelenik a kosárban, visszajelző üzenet. Eredmény: SIKERES ✓

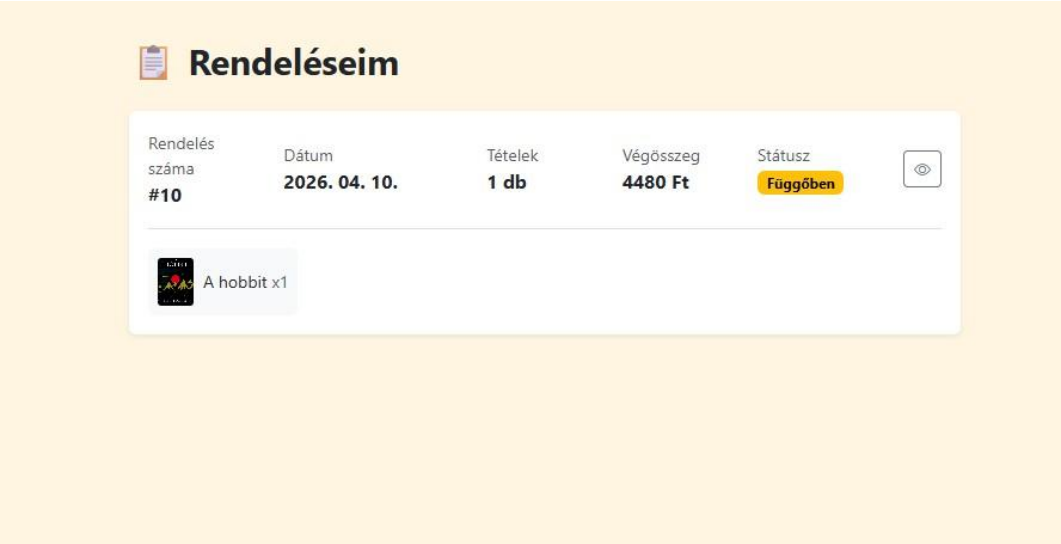


10. ábra:Sikeres visszajelző üzenet



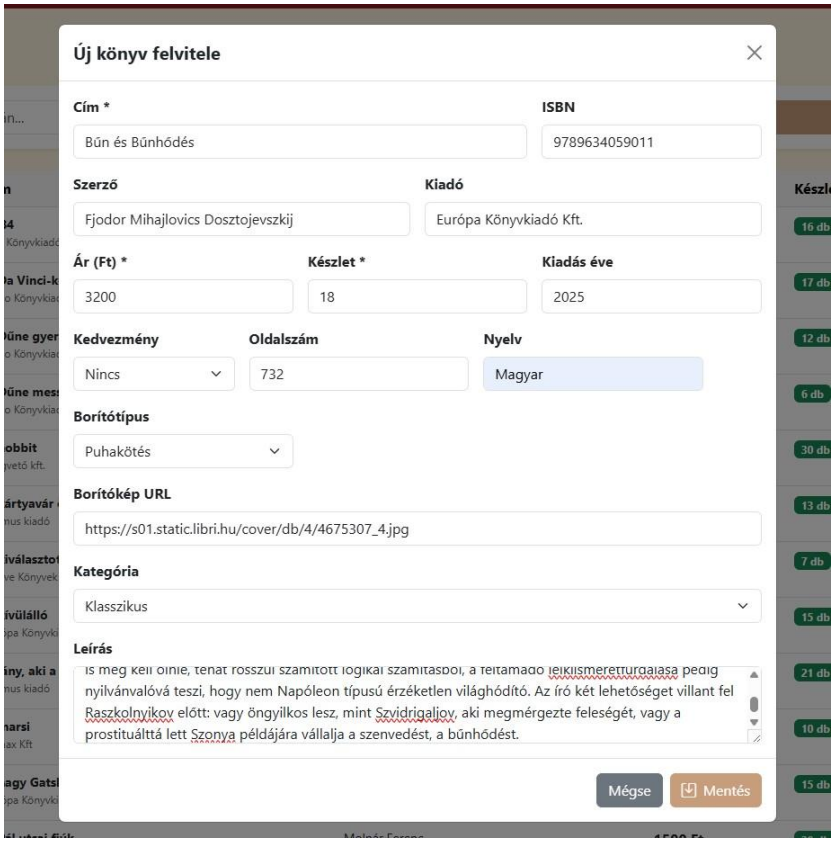
11. ábra:Könyv megjelenése a kosárban

- **4. teszteset – Rendelés leadása:** Kosár feltöltése után szállítási adatok megadása és rendelés leadása. Várt eredmény: rendelés rögzítése, kosár ürítése. Eredmény: SIKERES ✓



12. ábra:Rögzített rendelés

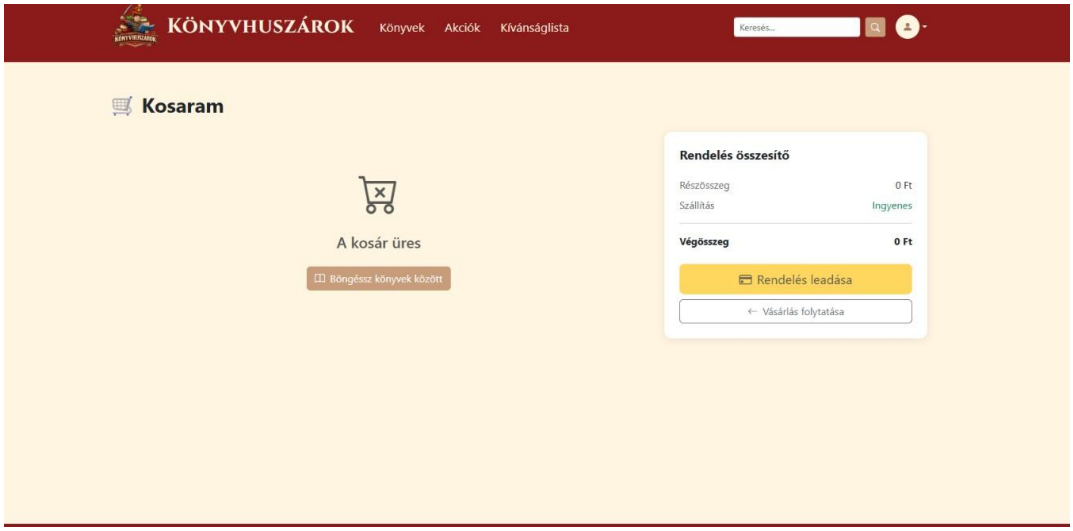
- **5. teszteset – Admin könyv felvétel:** Admin belépés után új könyv létrehozása kötelező mezők kitöltésével. Várt eredmény: könyv megjelenik a listában. Eredmény: SIKERES ✓



13. ábra:Könyv szerkesztő modal

- **6. teszteset – Üres kosár rendelés (extrém eset):** Rendelés próbálkozás üres kosárral. Várt eredmény: nem indítható rendelés.

  Eredmény: SIKERES ✓



14. ábra:Rendelés leadása oldal üres kosárral

- **7. teszteset – Admin védelme (extrém eset):** Admin oldal elérése bejelentkezés nélkül. Várt eredmény: átirányítás a bejelentkezés oldalra.

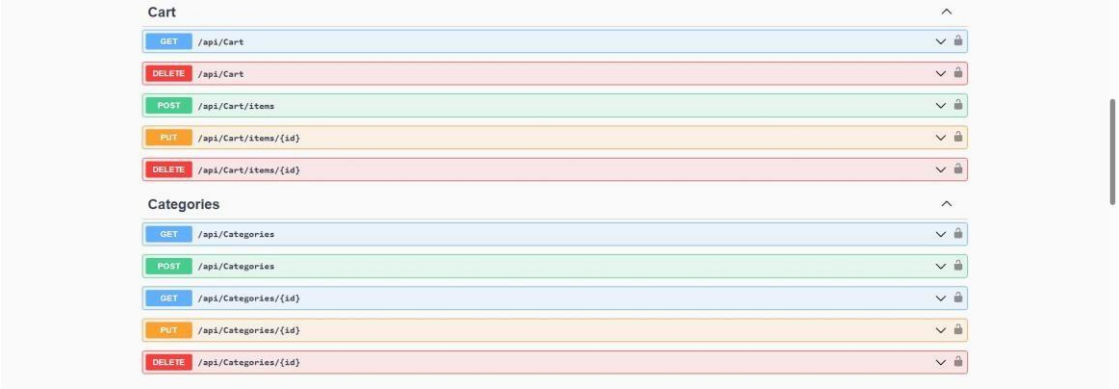
  Eredmény: SIKERES ✓

Fehér doboz tesztelés

A Swagger felületen keresztül közvetlenül teszteltük az API végpontokat. Ellenőriztük a HTTP válaszkódokat (200 OK, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found), a JSON válaszok formátumát, valamint a jogosultság-kezelést. A tesztelés során meggyőződtünk arról, hogy az adminisztrátori végpontok valóban csak érvényes admin tokennel érhetők el, és hogy a beviteli validáció megfelelően működik.

15. ábra:Swagger API dokumentáció – Addresses, Auth, Books végpontok

A Cart és Categories végpontok:

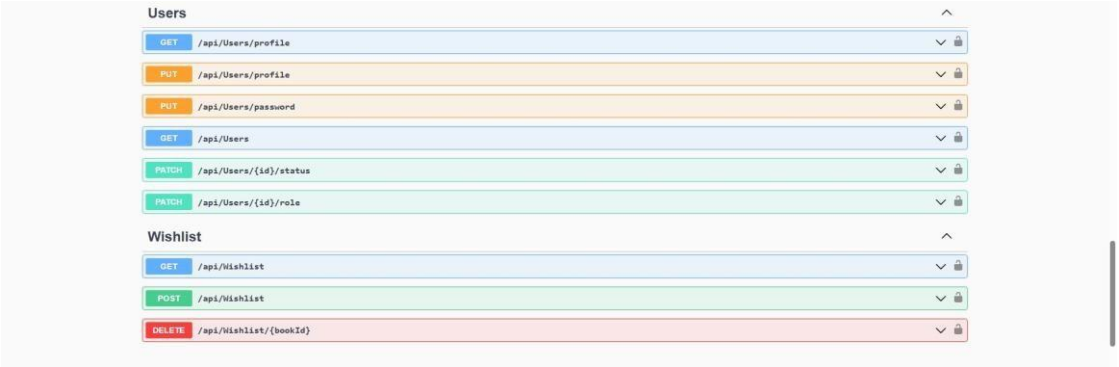


16. ábra:Swagger API dokumentáció – Cart, Categories végpontok

A Discounts, Orders és Reviews végpontok:

17. ábra:Swagger API dokumentáció – Discounts, Orders, Reviews végpontok

A Users és Wishlist végpontok:



18. ábra:Swagger API dokumentáció – Users, Wishlist végpontok

A tesztelés során feltárt hibák és javítások:

- Könyv szerkesztésekor a kategória nem töltődött vissza a legördülő listába – javítva
- Kategória módosításakor a régi kategória is megmaradt – javítva DELETE végpont hozzáadásával
- Kiadó mező nem mentődött el szerkesztéskor – javítva a BooksController GetBooks metódusában
- Bejelentkezésnél a hibás jelszó esetén megjelenő hibaüzenet azonnal eltűnt az oldalon – javítva az apiCall függvényben a 401-es státuszkód kezelésének módosításával, hogy a bejelentkezési végponton ne irányítson át automatikusan
- A könyvek szűrésekor a kategória rádiógomb kiválasztása után a szűrés nem azonnal érvényesült, hanem csak késleltetve – javítva a setTimeout eltávolításával és async/await alkalmazásával
- A navbar keresőmezőjében való keresés után a Könyvek oldalon az előzőleg beállított kategóriaszűrő megmaradt – javítva a localStorage booksFilter törlésével navigáció előtt
- Az akciós könyvek dátumkezelésekor időzóna-különbség miatt a kedvezmény nem jelent meg – javítva DateTime.Now használatával a DateTime.UtcNow helyett
- A kosár oldal lábléce kisebb tartalom esetén nem az oldal alján jelent meg – javítva flexbox sticky footer megoldással
- Mobilon egyes kijelzőméreteken az oldal jobb oldalán üres sáv jelent meg – javítva overflow-x: hidden CSS tulajdonsággal
- Az admin könyv szerkesztő modalban a kedvezmény mezők szerkesztés után nem töltődtek vissza helyesen az időzóna-különbség miatt – javítva a toLocalInput segédfüggvénnyel
- A főoldali kategória gombok használatakor minimális késleltetés volt megfigyelhető a szűrés beállításánál – javítva a setTimeout helyett await használatával a readUrlParams függvényben
- Az Akciók és Kívánságlista oldalakról visszanavigálva a könyv kategóriaszűrő megmaradt – javítva az auth.js DOMContentLoaded eseménykezelőjében
- A visszaállítás gomb a szűrési feltételeket visszaállította, de az új szűrést nem alkalmazta azonnal – javítva a loadBooks azonnali meghívásával

- A mobilos navbar logója és felirata egyes kijelzőkön lelógott a képernyőről – javítva media query és kisebb betűméret alkalmazásával
- Kisebb tartalmú oldalakon a lábléc nem az oldal alján helyezkedett el – javítva flexbox sticky footer megoldással
- Mobilon egyes kijelzőméreteken az oldal jobb oldalán üres sáv jelent meg – javítva overflow-x: hidden CSS tulajdonsággal
- A kedvezmény dátummezők szerkesztés után nem töltődtek vissza helyesen az időzóna-különbség miatt – javítva toLocalInput segédfüggvénnyel
- Az akciós könyvek nem jelentek meg az időzóna-különbség miatt – javítva DateTime.Now használatával a DateTime.UtcNow helyett
- Az adatbázisban felesleges táblák maradtak (authors, publishers, invoices stb.) – eltávolítva phpMyAdmin segítségével és a migrációk frissítésével
- A clean code átszervezés során a Controllers rétegből kiszervezett üzleti logika a Services rétegbe került, amelynek eredményeképpen a request osztályok a DTOs mappába lettek áthelyezve az egységes kódszervezés érdekében
- A utils.js fájl bevezetése előtt a showToast(), addToCart() és createBookCard() függvények minden JavaScript fájlban külön-külön szerepeltek, ezáltal megsértve a DRY elvet – javítva közös utils.js fájl létrehozásával
- A book-detail oldalról visszanavigálva elveszett az előzőleg beállított szűrési állapot – javítva localStorage alapú szűrőmentéssel
- Az admin felületen a felhasználók listájában a szerepkör módosítása után az oldal nem frissült automatikusan – javítva az adatok újratöltésével mentés után
- A könyv részletes oldalán az értékelés elküldése után az értékelések listája nem frissült automatikusan – javítva az értékelések újratöltésével sikeres mentés után
- A checkout oldalon az alapértelmezett szállítási cím nem töltődött be automatikusan – javítva az alapértelmezett cím lekérésével és automatikus betöltésével

IV. Felhasználói dokumentáció

1. A program általános specifikációja, célja

A Könyvhuszárok egy modern, webalapú online könyvesbolt alkalmazás, amelyet bárki használhat, aki könyveket szeretne vásárolni az interneten keresztül. Az alkalmazás célja, hogy egy könnyen kezelhető, átlátható és biztonságos vásárlási élményt nyújtson a felhasználóknak, miközben az adminisztrátoroknak teljes körű eszközöket biztosít a termékek, rendelések és felhasználók kezeléséhez.

Főbb funkciók felhasználóknak:

- Regisztráció és bejelentkezés – biztonságos, JWT token alapú authentikációval
- Könyvböngészés – szűrés kategória, szerző és ár szerint, rendezési lehetőségekkel
- Kosárkezelés – könyvek hozzáadása, mennyiség módosítása, törlése
- Rendelés leadása – szállítási cím és fizetési mód megadásával
- Rendelési előzmények – korábbi rendelések megtekintése és státusza
- Profil szerkesztése – személyes adatok és jelszó módosítása
- Kívánságlista – könyvek mentése későbbi megtekintéshez
- Könyvértékelések – 1-5 csillagos értékelés és szöveges vélemény írása
- Akciók oldal – aktuálisan kedvezményes könyvek megtekintése

2. Rendszerkövetelmények

Szerver oldali követelmények (backend futtatásához):

- .NET 8.0 Runtime vagy újabb
- MySQL 8.0 adatbázis-szerver (XAMPP segítségével telepíthető)
- Minimálisan 4 GB RAM, ajánlott 8 GB
- Windows 10/11, Linux vagy macOS operációs rendszer
- Legalább 500 MB szabad lemezterület

Kliens oldali követelmények (böngészőhöz):

- Modern webböngésző: Google Chrome 90+, Mozilla Firefox 88+, Microsoft Edge 90+, Safari 14+
- JavaScript engedélyezve a böngészőben
- Internetkapcsolat (vagy helyi hálózat a backend eléréséhez)
- Minimálisan 1366×768 pixeles felbontás ajánlott asztali nézethez

3. A program telepítése

Az alkalmazás futtatásához az alábbi lépéseket kell elvégezni sorban:

1. lépés – XAMPP telepítése és MySQL indítása

- Töltse le és telepítse a XAMPP-ot a https://www.apachefriends.org oldalról
- Indítsa el a XAMPP Control Panel-t és kattintson a MySQL melletti Start gombra
- Nyissa meg a phpMyAdmin felületet a http://localhost/phpmyadmin/ címen

2. lépés – Adatbázis importálása

- A phpMyAdmin bal oldali menüjében kattintson az Új gombra és hozzon létre egy konyvhuszarok nevű adatbázist
- Kattintson az Importálás fülre, majd válassza ki a mellékelt konyvhuszarok.sql fájlt
- Kattintson az Indítás gombra – az adatbázis és az összes tábla automatikusan létrejön

3. lépés – Backend indítása

- Nyissa meg a Konyvhuszarok.sln fájlt Visual Studio-ban (2022 vagy újabb)
- Ellenőrizze az appsettings.json fájlban a kapcsolati karakterláncot – alapból localhost:3306, felhasználó: root, jelszó: üres
- Nyomja meg az F5 billentyűt vagy kattintson a Start gombra a backend elindításához
- A backend elindul és automatikusan megnyílik a Swagger felület a https://localhost:7173/swagger/ címen

4. lépés – Frontend megnyitása

- Nyissa meg a böngészőt és látogasson el a https://localhost:7173/index.html oldalra
- Az adminisztrátori fiók alapértelmezett adatai: email: admin@konyvhuszarok.hu, jelszó: Admin1234!

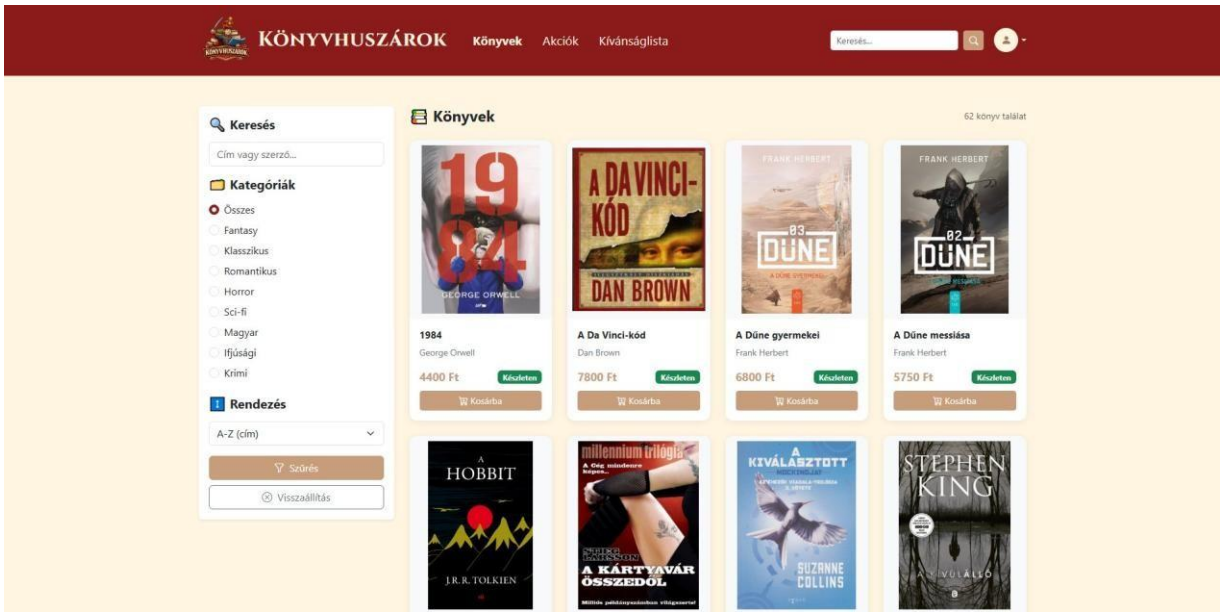
4. A program használatának részletes leírása

Regisztráció és bejelentkezés

Az alkalmazás megnyitásakor a főoldalon a jobb felső sarokban található profil ikonra kattintva érhető el a Bejelentkezés és Regisztráció opció. A regisztrációhoz meg kell adni a teljes nevet, email-címet, jelszót és opcionálisan telefonszámot. A jelszónak legalább 6 karakterből kell állnia. Sikeres regisztráció után azonnal bejelentkezhet a megadott adatokkal.

Könyvek böngészése és keresés

A Könyvek oldalon megtekinthető az összes elérhető könyv. A bal oldali sávban kategória szerint szűrhet, az oldal tetején keresőmezőt és rendezési lehetőséget talál. A szűrők kombinálhatók: egyszerre lehet kategória, keresőszó és rendezési szempont szerint szűkíteni a listát. A könyv kártyáján látható a cím, szerző, ár, és ha van akciós ár, akkor az eredeti ár áthúzva és az akciós ár pirossal kiemelve jelenik meg. A könyv nevére kattintva megnyílik a részletes nézet, ahol az összes adat, az értékelések és a kosárba adás gomb található.



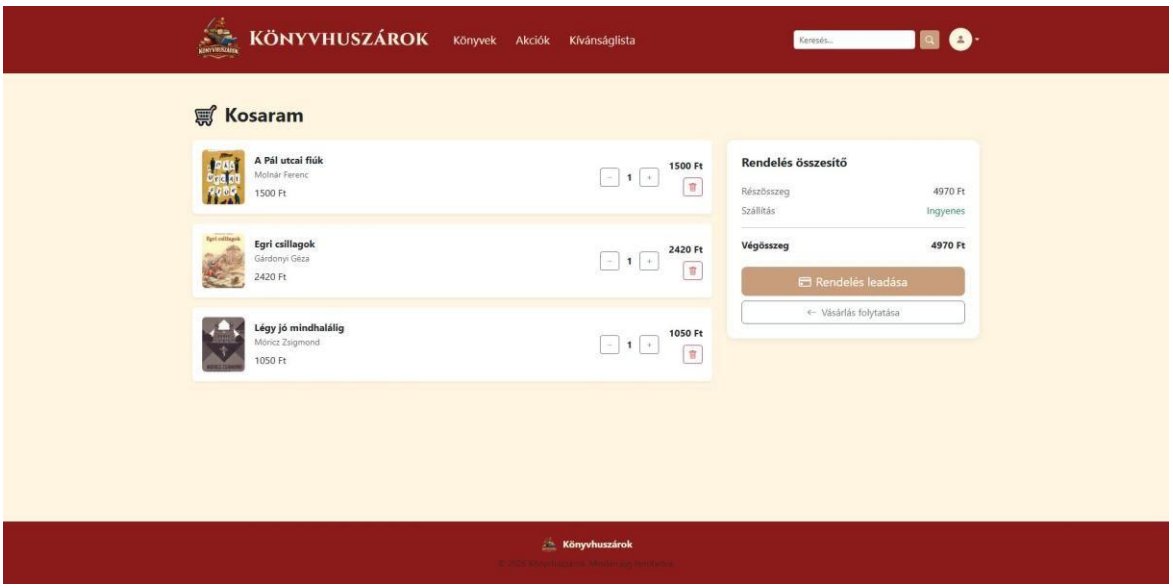
19. ábra:A Könyvek oldal szűrési lehetőségekkel

Az alábbi képen a könyv részletes oldala látható, ahol az összes könyvadaton (ISBN, kiadó, oldalszám, nyelv, kiadás éve) kívül a leírás, az értékelések és a kosárba adás gomb is megtalálható:

20. ábra:Könyv részletes oldal

Kosár és rendelés leadása

Bejelentkezés után a könyv kártyáján lévő Kosárba gombra kattintva a könyv bekerül a kosárba. A kosár tartalmát a Kosár menüpontban lehet megtekinteni, ahol módosítható a mennyiség és törölhetők a tételek. A rendelés leadásához kattintson a Megrendelés gombra, adja meg a szállítási címet és a fizetési módot, majd erősítse meg a rendelést. Sikeres rendelés után a rendszer visszajelzést küld és a kosár kiürül.



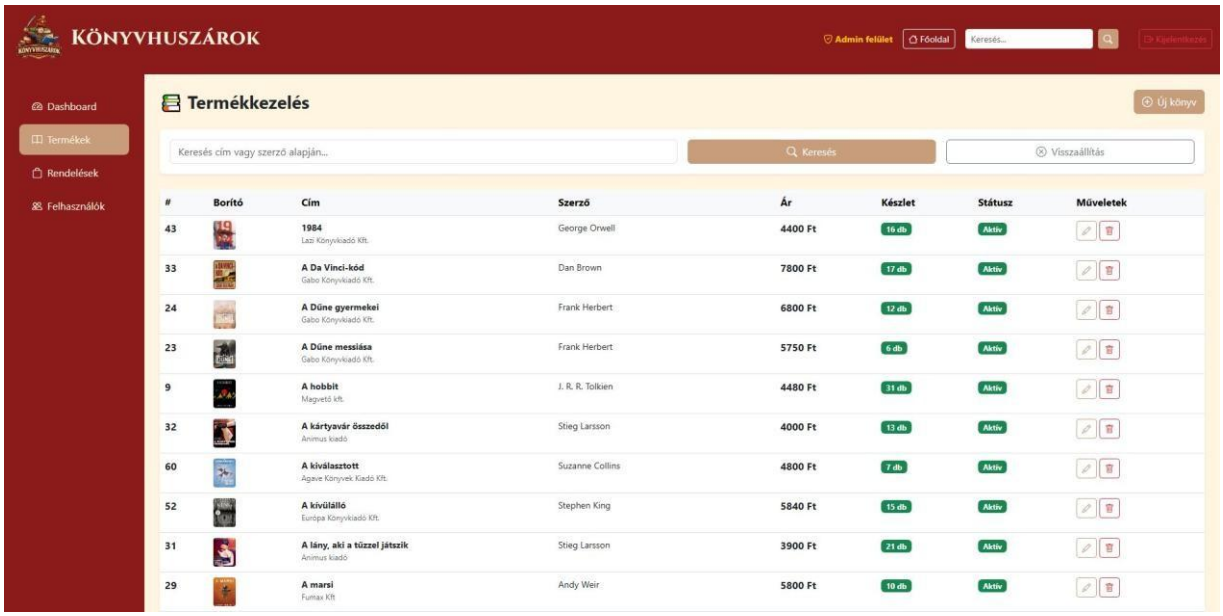
21. ábra:A kosár oldal

Profil kezelése

A profil oldalon szerkeszthető a felhasználó neve és telefonszáma, illetve megváltoztatható a jelszó. Az oldalon mentett szállítási címek is kezelhetők – hozzáadhatók új címek, és beállítható az alapértelmezett cím, amely a rendelésnél automatikusan betöltődik.

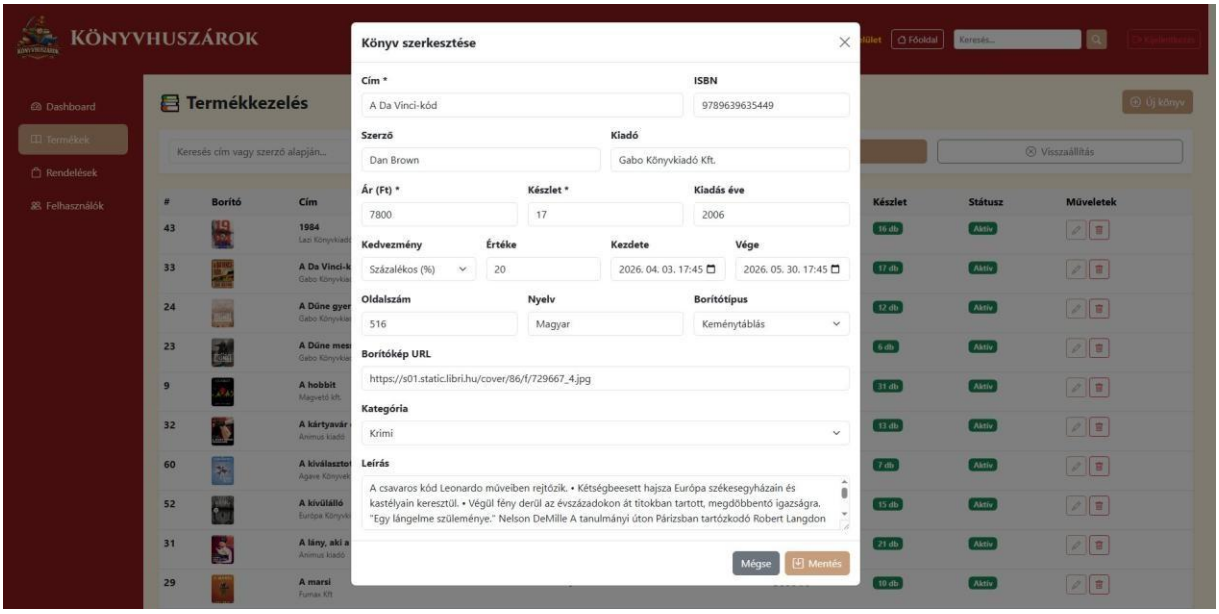
Adminisztrátori felület

Az adminisztrátori felület a profil legördülő menüből érhető el, kizárólag admin szerepkörű felhasználóknak. Az admin felület négy fő részből áll: Dashboard (statisztikák és összesítők), Termékek (könyvek kezelése), Rendelések (rendelések státuszának frissítése) és Felhasználók (felhasználók kezelése). A Termékek oldalon új könyv adható hozzá, és a meglévő könyvek szerkeszthetők, törölhetők. Könyv szerkesztésekor lehetőség van kedvezmény beállítására is: a típus (százalékos vagy fix összeg), az érték és az érvényességi időszak megadásával. A kedvezményes könyvek az Akciók oldalon jelennek meg a vásárlóknak.



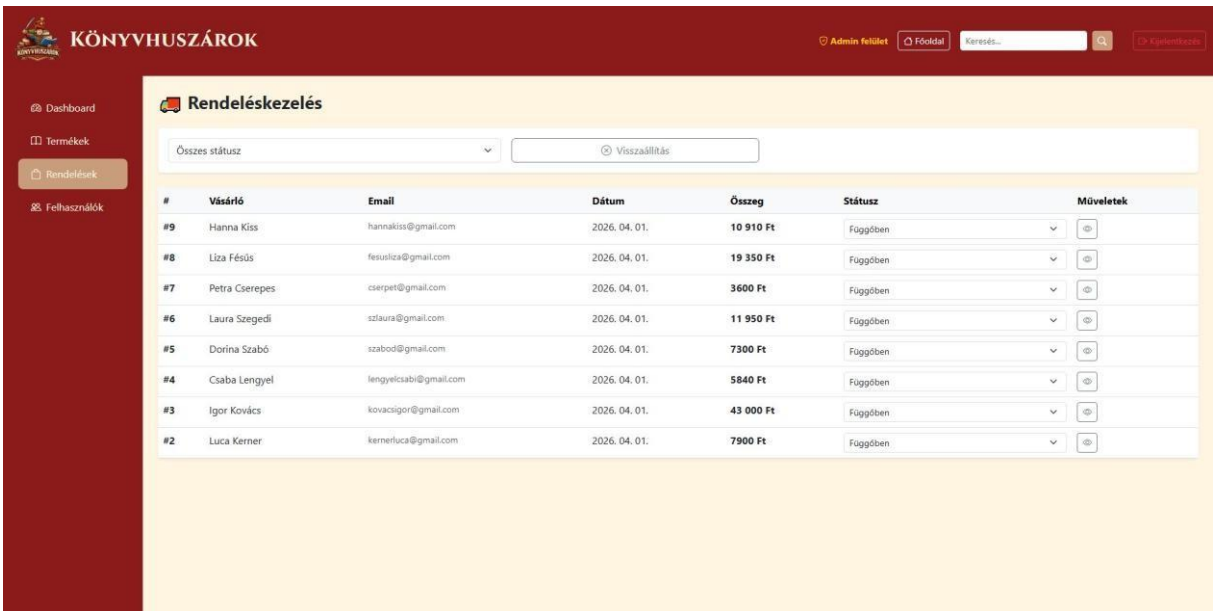
22. ábra:Admin termékkezelés oldal

Könyv hozzáadásakor vagy szerkesztésekor egy felugró modal ablak jelenik meg az összes szerkeszthető mezővel, köztük a kedvezmény beállítással:



23. ábra:Könyv szerkesztő modal kedvezmény beállítással

A Rendelések oldalon az összes leadott rendelés megtekinthető és kezelhető. Az adminisztrátor a státusz legördülő menüből tudja frissíteni a rendelés állapotát:

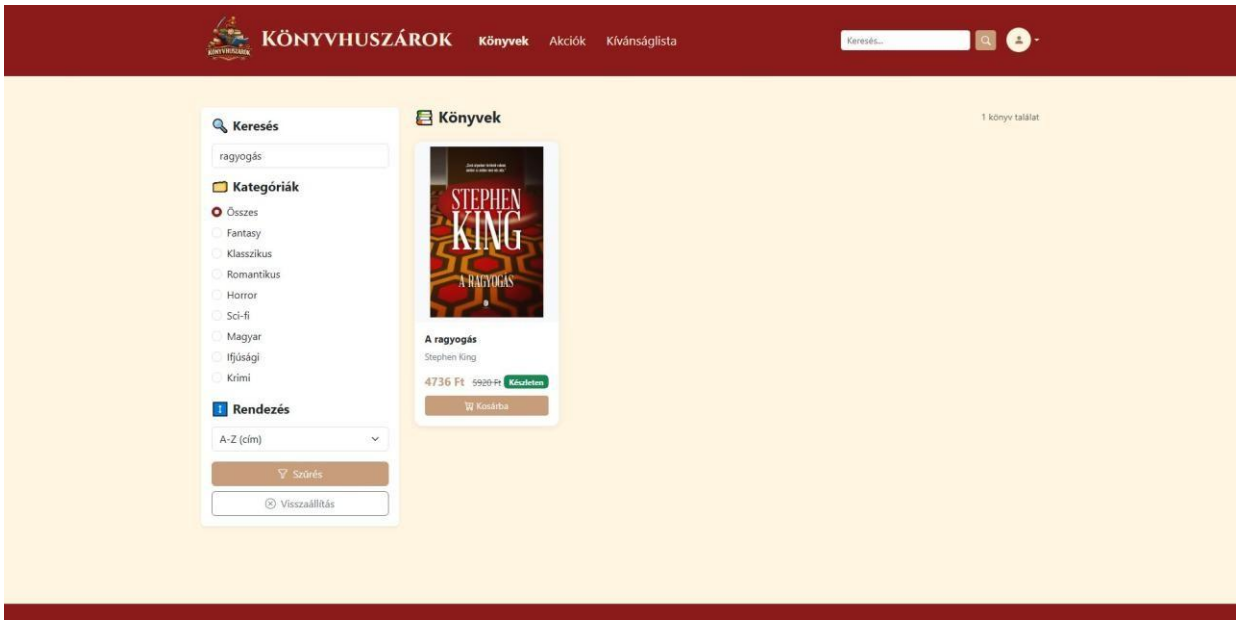


24. ábra:Admin rendeléskezelés oldal

Az alábbi képen látható, hogyan jelenik meg a kedvezményes ár a könyv részletes nézetén – az akciós ár kiemelve, az eredeti ár áthúzva szerepel az Akció! felirat mellett:

25. ábra:Akciós könyv részletes nézete

A könyv kártyán is megjelenik az akciós ár: az eredeti ár áthúzva, mellette a kedvezményes ár látható:



26. ábra:Akciós könyv kártyán megjelenő kedvezmény

V. Összegzés

A Könyvhuszárok projekt fejlesztése során sikeresen megvalósítottunk egy teljes funkcionalitású online könyvesbolt webalkalmazást. A kitűzött célokat elértük: az alkalmazás lehetővé teszi a könyvek böngészését, vásárlását, a rendelések nyomon követését, valamint az adminisztrátorok számára a teljes körű tartalomkezelést.

A fejlesztés során számos technikai kihívással kellett szembenéznünk, amelyek megoldása komoly tapasztalatot adott. A JWT token alapú authentikáció megvalósítása, az Entity Framework Core Code-First megközelítés alkalmazása, a reszponzív felhasználói felület kialakítása, valamint a clean code elvek szerinti kódszervezés mind-mind olyan területek, ahol jelentős fejlődést értünk el a fejlesztési folyamat során.

A projekt során különösen értékes tapasztalatot szereztünk a RESTful API tervezés és a kliens-szerver kommunikáció területén. A Services és DTOs rétegek bevezetésével a kód átláthatóbbá és karbantarthatóbbá vált, ami a clean code elvek gyakorlati alkalmazását demonstrálja. A közös JavaScript segédfüggvények (utils.js) kiszervezésével a frontend kódban is megvalósult a DRY elv.

Jövőbeli fejlesztési lehetőségek:

- Online fizetési rendszer integrálása (pl. Stripe vagy PayPal)
- Email értesítések küldése rendelés leadásakor és státuszváltozáskor
- Könyvajánló rendszer a vásárlási előzmények alapján
- Natív mobilalkalmazás fejlesztése (Android/iOS)
- Kuponkód rendszer bevezetése
- Részletesebb adminisztrátori statisztikák és exportálási lehetőségek

Összességében a projekt sikeresen teljesítette a vizsgaremekkel szemben támasztott összes követelményt: megvalósult a RESTful architektúra, a kliens és szerver oldali komponensek szétválasztása, az adatbázis-kezelés, a reszponzív megjelenítés és a clean code elvek szerinti kódszervezés. A fejlesztés során megszerzett tapasztalatok és ismeretek értékes alapot nyújtanak a jövőbeli szoftverfejlesztési munkákhoz.

VI. Ábrajegyzék

1. ábra:A Könyvhuszárok főoldala 4
2. ábra:A Könyvhuszárok adatbázis diagramja 10
3. ábra: A projekt mappastruktúrája Visual Studio Solution Explorerben 10
4. ábra:A LoginAsync metódus forráskódja (AuthService.cs) 11
5. ábra:A CreateOrderAsync metódus forráskódja (OrderService.cs) 12
6. ábra:A GetBooks végpont forráskódja (BooksController.cs) 13
7. ábra:Rendelés leadása oldal 14
8. ábra:Profil oldal 14
9. ábra:Sikeres regisztráció 15
10. ábra:Sikeres visszajelző üzenet 16
11. ábra:Könyv megjelenése a kosárban 16
12. ábra:Rögzített rendelés 17
13. ábra:Könyv szerkesztő modal 17
14. ábra:Rendelés leadása oldal üres kosárral 18
15. ábra:Swagger API dokumentáció – Addresses, Auth, Books végpontok 18
16. ábra:Swagger API dokumentáció – Cart, Categories végpontok 19
17. ábra:Swagger API dokumentáció – Discounts, Orders, Reviews végpontok 19
18. ábra:Swagger API dokumentáció – Users, Wishlist végpontok 19
19. ábra:A Könyvek oldal szűrési lehetőségekkel 24
20. ábra:Könyv részletes oldal 25
21. ábra:A kosár oldal 25
22. ábra:Admin termékkezelés oldal 26
23. ábra:Könyv szerkesztő modal kedvezmény beállítással 27
24. ábra:Admin rendeléskezelés oldal 27
25. ábra:Akciós könyv részletes nézete 28
26. ábra:Akciós könyv kártyán megjelenő kedvezmény 28

VII. Irodalomjegyzék, hivatkozásjegyzék

- Microsoft. (2024). ASP.NET Core documentation. https://docs.microsoft.com/en-us/aspnet/core
- Microsoft. (2024). Entity Framework Core documentation. https://docs.microsoft.com/en-us/ef/core
- Bootstrap. (2024). Bootstrap 5 documentation. https://getbootstrap.com/docs/5.3
- MySQL. (2024). MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en
- JWT.io. (2024). JSON Web Tokens. https://jwt.io/introduction
- Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.
- dbdiagram.io. (2024). Database diagram tool. https://dbdiagram.io
- Swashbuckle. (2024). Swagger/OpenAPI documentation for ASP.NET Core. https://github.com/domaindrivendev/Swashbuckle.AspNetCore